

CORS Fix - Production Deployment Guide

IMMEDIATE ACTION REQUIRED

Your production backend needs environment variable updates to fix the CORS error preventing images from loading on <https://kidrove.com>.

Backend Environment Variables (Production Server)

Where to update: Your backend hosting platform (Render, Railway, Heroku, etc.) environment variables

Required Updates

Add or update these environment variables on your production backend:

```
# Primary frontend URL (REQUIRED)
FRONTEND_URL=https://kidrove.com

# Additional allowed origins (OPTIONAL - if you need www subdomain)
ADDITIONAL_ALLOWED_ORIGINS=https://www.kidrove.com

# Verify your Cloudinary cloud name is set (REQUIRED for directUrl optimization)
CLOUDINARY_CLOUD_NAME=your_cloudinary_cloud_name

# Verify your BASE_URL is correct (REQUIRED for API URLs)
BASE_URL=https://api.kidrove.com
```

Step-by-Step Instructions

For Render.com:

1. Go to your dashboard: <https://dashboard.render.com>
2. Select your backend service (api.kidrove.com)
3. Click on "Environment" in the left sidebar
4. Add/Update environment variables:
 - Key: `FRONTEND_URL`, Value: `https://kidrove.com`
 - Key: `ADDITIONAL_ALLOWED_ORIGINS`, Value: `https://www.kidrove.com`
5. Click "Save Changes"
6. The service will automatically redeploy

For Railway.app:

1. Go to your project dashboard
2. Click on your backend service
3. Go to "Variables" tab
4. Add/Update environment variables:
 - `FRONTEND_URL=https://kidrove.com`
 - `ADDITIONAL_ALLOWED_ORIGINS=https://www.kidrove.com`
5. Click "Deploy" or wait for automatic redeploy

For Heroku:

1. Go to your app dashboard
2. Click "Settings" tab
3. Click "Reveal Config Vars"
4. Add/Update:
 - Key: `FRONTEND_URL`, Value: `https://kidrove.com`
 - Key: `ADDITIONAL_ALLOWED_ORIGINS`, Value: `https://www.kidrove.com`
5. The app will automatically restart

For Custom VPS/Server:

1. SSH into your server
2. Edit your `.env` file:

```
nano /path/to/your/backend/.env
```

3. Add/Update these lines:

```
FRONTEND_URL=https://kidrove.com
ADDITIONAL_ALLOWED_ORIGINS=https://www.kidrove.com
CLOUDINARY_CLOUD_NAME=your_cloudinary_cloud_name
BASE_URL=https://api.kidrove.com
```

4. Restart your backend service:

```
pm2 restart backend
# OR
systemctl restart your-backend-service
```

Frontend Environment Variables (Production)

Where to update: Your frontend hosting platform (Vercel, Netlify, etc.) environment variables

Required Updates

Ensure these are set correctly on your production frontend:

```
# For SEO and URL generation
VITE_APP_URL=https://kidrove.com

# For API calls (use either variable name)
VITE_API_URL=https://api.kidrove.com/api
# OR
VITE_API_BASE_URL=https://api.kidrove.com/api
```

Step-by-Step Instructions

For Vercel:

1. Go to your project dashboard
2. Click "Settings" → "Environment Variables"
3. Add/Update:
 - Key: `VITE_APP_URL`, Value: `https://kidrove.com`
 - Key: `VITE_API_URL`, Value: `https://api.kidrove.com/api`
4. Redeploy your frontend:
 - Go to "Deployments" tab
 - Click "..." on latest deployment → "Redeploy"

For Netlify:

1. Go to "Site settings" → "Environment variables"
2. Add/Update:
 - `VITE_APP_URL=https://kidrove.com`
 - `VITE_API_URL=https://api.kidrove.com/api`
3. Trigger new deployment:
 - Go to "Deploys" tab
 - Click "Trigger deploy" → "Deploy site"

Testing After Deployment

1. Test CORS in Browser Console

Open your browser console on `https://kidrove.com` and run:

```
// Test 1: Fetch a media file
fetch('https://api.kidrove.com/api/media/file/35caf6df-cba4-4c92-8395-125afd64331c')
  .then(r => {
    console.log('✅ CORS working! Status:', r.status);
    return r.blob();
  })
  .then(blob => console.log('✅ Image loaded:', blob.size, 'bytes'))
  .catch(err => console.error('❌ CORS error:', err));

// Test 2: Check if images load
const img = new Image();
img.onload = () => console.log('✅ Image loaded successfully');
img.onerror = () => console.error('❌ Image failed to load');
img.src = 'https://api.kidrove.com/api/media/file/35caf6df-cba4-4c92-8395-125afd64331c';
```

2. Verify in Network Tab

1. Open browser DevTools (F12)
2. Go to "Network" tab
3. Reload the page
4. Look for media file requests
5. Check response headers should include:

```
access-control-allow-origin: https://kidrove.com
```

3. Check for Errors

In the Console tab, you should see **NO** errors like:

- ❌ "Not allowed by CORS"
- ❌ "CORS policy: No 'Access-Control-Allow-Origin' header"

What Changed in the Code

Backend Changes

File: backend/src/models/MediaAsset.ts

Added a virtual field `directUrl` that:

- Returns Cloudinary CDN URL directly for Cloudinary assets (faster, no backend proxy)
- Falls back to backend API URL for local files
- Automatically included in all API responses (toJSON enabled)

Example Response:

```
{
  "_id": "...",
  "uuid": "35caf6df-cba4-4c92-8395-125afd64331c",
  "url": "https://api.kidrove.com/api/media/file/35caf6df-cba4-4c92-8395-125afd64331c",
  "directUrl": "https://res.cloudinary.com/your-cloud/image/upload/v123/folder/image.jpg",
  "provider": "cloudinary",
  "publicId": "folder/image",
  ...
}
```

Frontend Changes

File: frontend/src/config.ts

Updated to support both environment variable naming conventions:

- `VITE_API_URL` (new, preferred)
- `VITE_API_BASE_URL` (legacy, backwards compatible)

Using the New `directUrl` Field (Optional)

Once backend environment variables are updated and the backend is redeployed, the `directUrl` field will be available in all media asset responses.

Frontend Components (Future Optimization)

You can optionally update frontend components to use `directUrl` for better performance:

```
// Example: In any component using media assets

// Before (slower - proxies through backend)
const imageUrl = `${config.apiUrl}/media/file/${asset.uuid}`;

// After (faster - uses Cloudinary CDN directly)
const imageUrl = asset.directUrl || asset.url || `${config.apiUrl}/media/file/${asset.uuid}`;
```

Benefits of using `directUrl`:

- ☒ Faster image loading (CDN edge servers)
- ☒ No CORS issues (Cloudinary handles CORS)
- ☒ Reduced backend server load
- ☒ Free image transformations via URL params
- ☒ Global CDN caching

When to implement this:

- After confirming CORS fix works
- During next frontend update cycle
- Not urgent - current fix resolves the CORS issue

Rollback Plan (If Issues Occur)

If you encounter any issues after updating environment variables:

Backend Rollback:

1. Remove or revert the `FRONTEND_URL` variable
2. Restart the backend service
3. Images will require alternative hosting solution

Frontend Rollback:

1. Frontend changes are backwards compatible
2. No rollback needed - code works with or without the new env vars

Support Checklist

- ☒ Backend environment variables updated
- ☒ Backend service restarted/redeployed
- ☒ Frontend environment variables verified
- ☒ Frontend redeployed (if env vars changed)
- ☒ Tested CORS in browser console
- ☒ No CORS errors in Network tab
- ☒ Images loading correctly on production site

Additional Notes

Security

- The CORS configuration explicitly allows only your frontend domain
- Media files still check `isPublic` flag before serving
- Credentials (cookies, auth headers) are enabled

Performance

- Cloudinary assets now served directly from CDN (when using `directUrl`)
- Backend API URL still works as fallback
- Both URLs are valid and functional

DNS Considerations

If you're using both `kidrove.com` and `www.kidrove.com`:

- Add both domains to `ADDITIONAL_ALLOWED_ORIGINS` (comma-separated)
 - Set up proper DNS redirects (301) to your preferred domain
 - Cloudinary URLs work from both domains
-

Questions?

If you encounter any issues:

1. Check backend logs for CORS rejection messages
2. Verify environment variables are set correctly
3. Ensure backend service restarted after env var changes
4. Test with browser console commands provided above
5. Check that `CLOUDINARY_CLOUD_NAME` matches your Cloudinary account

Common Issues:

- **Still seeing CORS errors:** Backend might not have restarted after env var update
- **Images not loading:** Check `CLOUDINARY_CLOUD_NAME` is correct
- **API calls failing:** Verify `FRONTEND_URL` is exactly `https://kidrove.com` (no trailing slash)